Start-Controlled Evaluation of JEPA Controllers: Test-Time Search Collapses on Manipulation, and Amortization Needs a Horizon It Should Infer

Anonymous Author(s)
Affiliation
Address
email

Abstract

Forming behavioral policies from pretrained JEPA representations requires a test-time procedure for sequential decision making, typically sampling-based search or model predictive control (MPC). Recent work claims that amortized goal-conditioned inverse dynamics (GC-IDM) offers a cheaper alternative that matches or beats this search. We propose a start-controlled evaluation protocol for JEPA-based controllers on the existing benchmark suite, and we identify and remove a deployment-critical assumption the amortized policy relies on. Start-controlled evaluation measures from the true episode start rather than a random mid-episode state. Our key finding is that test-time search collapses on the manipulation task under the standard latent-distance planning cost. Random-start evaluation instead inflates success rates unevenly across tasks and can even reverse in sign. We then show that the amortized policy's performance depends on privileged information. GC-IDM is handed the ground-truth temporal distance to the target to guide action generation, information a deployed system rarely has (e.g. when goals come from a hierarchical sub-goal generator). We propose VC-IDM, which removes this dependence with a learned temporal-distance value, where a deployed system operates. Across four environments and two representations, VC-IDM wins when the target distance is unknown, variable, or beyond the training range. Unlike GC-IDM, it improves with additional budget rather than degrading. Evaluation methodology and conditioning mechanisms on pretrained representations change the conclusions drawn.

1 Introduction

Self-supervised, joint-embedding predictive architecture (JEPA) based world models learn environment dynamics in a latent space and let an agent reach goals by planning over latents [3, 4]. The representations they learn supply strong frozen latents for sequential decision making. LeWorldModel (LeWM) [7] trains a JEPA stably end-to-end from pixels with a next-embedding loss and an isotropic Gaussian regularizer (SIGReg), planning up to $48\times$ faster than foundation-model world models [7]. Sub-JEPA [16] sharpens this with subspace-Gaussian regularization and outperforms LeWM under the standard evaluation. The standard test-time control protocol is sampling-based search, namely CEM, iCEM, MPPI, and gradient-based MPC (SGD) [13, 12, 15], which rolls the predictor over action-sequence candidates and keeps the lowest-cost sequence. Search dominates the inference cost, requiring roughly $9{,}000$ candidate rollouts per planning step, as reported by Nguyen et al. [8].

In contrast to test-time search, and to avoid this planning tax, GC-IDM [8] proposes a learned goal-conditioned inverse-dynamics model trained and run on frozen latents, requiring only one forward pass per step to produce the action, $a = \mathrm{IDM}(z_t, z_g, h)$, conditioned on the current latent z_t, the goal latent z_g, and a remaining horizon h at environment step t. It reportedly matches or exceeds CEM

in seven of the eight environment–protocol cells it reports, at 100 to $130\times$ lower per-decision cost, and a broader sweep over planners indicates the result is not specific to one optimizer. Their thesis is that, in a sufficiently regularized latent space, the structure test-time search recovers is already locally encoded, allowing planning to shift from online optimization to learned inference.

Evaluating and visualizing LeWM and Sub-JEPA CEM policy rollouts, we find behavior contradicting the reported results. On the OGB-Cube pick task the gripper never approaches the cube, whereas GC-IDM behaves as its published numbers suggest. This raised the question of whether the evaluation fairly differentiates model performance. Both search and GC-IDM are scored from random mid-episode starts. We find this inflates success by dropping the agent close to the goal, sometimes into states the success criterion already counts as solved. We evaluate this within the official `stable-worldmodel` [6] ecosystem and evaluation harness, using the officially released LeWM and Sub-JEPA models and the GC-IDM source code.

We first find that start-controlled evaluation changes our understanding of amortization relative to test-time search once we evaluate from the true start ($t{=}0$). Random-start success is an artifact whose magnitude, and even sign, varies with the task. OGB-Cube and Push-T are inflated, Two-Room is near-neutral on Sub-JEPA yet inflated by up to $+14$ on LeWM, and Reacher is reversed. On the hardest task (OGB-Cube), the controlled start collapses every search method while GC-IDM still solves it. Second, we find that the amortized policy depends on privileged information. GC-IDM is told the ground-truth horizon and depends on it sharply, as both an inference condition and procedurally via the task-execution budget. The horizon quantity should be inferred, we remove the dependence with VC-IDM, which replaces the supplied horizon with a learned temporal-distance value.

Contributions.

- A start-controlled evaluation protocol for JEPA-based controllers on the existing benchmark suite, ensuring task success stems from the policy's actions and preventing the random-start performance inflation seen across tasks (§4). Under it, amortization uniquely solves OGB-Cube, outperforming test-time search methods that rely on crude latent-distance objectives.
- The identification of a deployment-critical dependence of the amortized policy. Its performance critically depends on privileged knowledge of the time-to-target, an assumption that breaks when the distance is unknown, as in realistic deployment scenarios (§5).
- VC-IDM, which removes the dependence by inferring the remaining temporal distance at each step, improving performance under unknown, variable, or out-of-range targets, and improves rather than degrades with budget (§6).

2 Related Work

JEPA world models learn dynamics in a compact embedding space that lets an agent plan over predicted latent trajectories [3, 4]. LeWM [7] trains a JEPA stably end-to-end from pixels with only a next-embedding prediction and a SIGReg loss term. SIGReg pulls the latent distribution toward an isotropic standard Gaussian without forming a full covariance. It draws many random unit directions, projects the embeddings onto them, and penalizes each 1-D marginal's deviation from $\mathcal{N}(0,1)$ through an Epps-Pulley goodness-of-fit statistic. By the Cramér-Wold argument, matching all random 1-D projections to a standard normal drives the joint latent toward isotropy, at a cost linear in the number of sketch directions. Sub-JEPA [16] applies SIGReg not to the full embedding but within K lower-dimensional subspaces (fixed orthonormal projections), where the projected-distribution test is lower-variance. Sub-JEPA outperforms LeWM on the standard benchmark suite. Regularizing subspaces rather than the whole space allows the global latent covariance to remain far from isotropic (§7). We use both as frozen encoders and neither propose a new world model nor fine-tune one.

Sampling-based MPC (CEM, iCEM, MPPI, and gradient-based trajectory optimization [13, 12, 15]), optionally paired with a learned terminal value [5], is the standard latent control protocol and where most inference cost resides [8]. GC-IDM [8] reformulates control as a latent inverse problem, replacing test-time search with a feed-forward inverse-dynamics model (IDM). IDMs have a long history, typically as proposal mechanisms or imitation components rather than the primary planner [1, 11, 2, 10]. We extend GC-IDM with a learned temporal distance to the goal, a quantity studied in

goal-conditioned RL as temporal-distance / quasimetric value learning [14]. Our use is minimal, namely a regression value that supplies the IDM's horizon, decoupling it from the test-time budget.

3 Setup and Reproduction

We work entirely within the OGBench and LeWM / Sub-JEPA ecosystem (`stable-worldmodel` [6]). Our critique and our fix are evaluated against the same models, data, and code. All components are the officially released artifacts (the published LeWM and Sub-JEPA encoder checkpoints and the GC-IDM source code for amortization). We alter only the evaluation protocol (§4) and the horizon conditioning (§5 and §6).

We use four single-objective goal-reaching tasks from the `stable-worldmodel` suite [6], namely OGB-Cube (3D block manipulation, action dimension 5), which comes from OGBench [9], Push-T (2D pushing, dim 2), Two-Room (2D navigation, dim 2), and Reacher (2D reaching, dim 2), which comes from the DeepMind Control Suite [**?**]. Following prior work, observations are images, and JEPA models are trained from offline expert demonstrations. A goal is a future frame of the same episode at a chosen offset.

Frozen JEPA encoders (LeWM, Sub-JEPA) map images to a d=192 latent ($\sim$ 15M-parameter encoders). We compare test-time search (CEM, iCEM, MPPI, SGD) with three amortized policies, namely GC-IDM under its original inference protocol, a horizon-free GC-IDM (no h), and VC-IDM (§6). In that protocol the horizon supplied at step t is $h = \min(T - t, H_{\max})$ following GC-IDM's released code. T is the target offset the controller is told, which also sets the episode budget, and $H_{\max}$ is the largest horizon seen during amortizer training. Architectures are detailed in §6. We report task success rate over 200 episodes per cell, mean $\pm$ std over 4 seeds. We evaluate across goal offsets (true target distances) $H \in \{50, 75, 100\}$, from the episode start (our start-controlled protocol, §4) and from a mid-episode start following prior work.

4 A Start-Controlled Evaluation Protocol

The comparison reported in prior work evaluates from a random mid-episode state, which our visual inspection shows hides the policy's true behavior. Analyzing the OGB-Cube dataset, we find that an episode is typically complete (the cube is at the target position) after about 75 of its 200 steps. A policy that does little can still post moderate success rates. On the search-based rollouts, the gripper never truly approaches the cube to pick it up. The solver succeeds only when (a) the cube already sits at the target, (b) the cube is already in hand and only a short transport remains, or (c) the gripper is initialized in a near-perfect pick pose. We re-evaluate every method from the controlled start (t=0) and measure the inflation, random minus t=0 (Table 1). Starting from t=0 ensures that success reflects what a human would qualitatively call success. The gripper must approach, grasp, and transport the cube. Figure 1 shows start-controlled rollouts for each task, contrasting a successful GC-IDM episode with the failing search solver.

We evaluate at goal offset 50 because success on OGB-Cube is defined by block position. At offset 25 every solver scores 100% because the block need not move, and a do-nothing policy is classed successful. Offset 50 forces meaningful displacement. OGB-Cube's large inflation (+59 for search) is the extreme of this confound. A random start drops the agent where the block is already near the goal. Push-T shows the same mechanism but milder (+24), while we observe deflation of up to -16 on Reacher (Table 1).

On OGB-Cube, all four search solvers score exactly 0.0 at t=0 for both representations, while GC-IDM is the only method reporting successes under both representations (Sub-JEPA 84.9, LeWM 68.9, Table 1). The solver success reported on OGB-Cube is therefore an evaluation artifact of random starts. The solvers function as expected and drive down the latent goal-cost they optimize, returning a low-cost plan, yet the executed behavior does not move the block to the goal. Figure 2 makes this concrete. On OGB-Cube the CEM optimizer drives the latent goal-cost it minimizes down by 99%, yet the arm never contacts the block. The true object-to-goal distance never falls (displacement 0.0 cm, 0% success). GC-IDM's latent goal-cost falls unevenly as the agent acts. Over the first 30 steps the latent goal-cost remains roughly constant as the gripper approaches the cube. After the cube is picked, it tracks the true block displacement step-for-step and the block reaches the

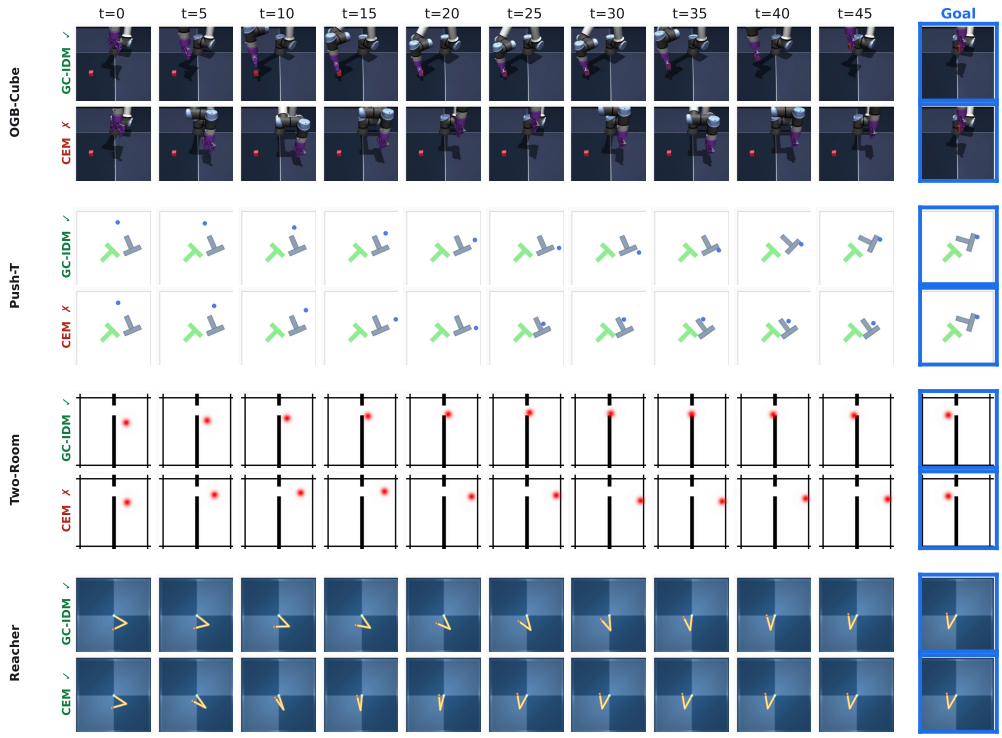


Figure 1: **CEM vs. GC-IDM rollouts under start-controlled evaluation.** For each task we select one random episode on which GC-IDM succeeds evaluated from t=0, using Sub-JEPA representations, a goal offset of 50. Observations shown every 5 steps with the goal at right. Crucially, CEM fails completely on OGB-Cube, where the gripper is never driven towards the cube to perform the pick, yet it succeeds on Reacher (representative of the complete results in Table 1).

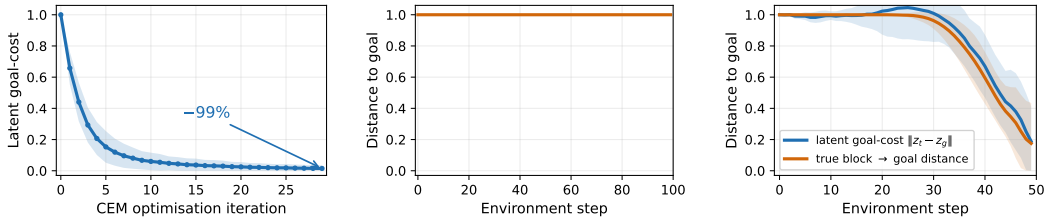
goal. Under CEM the cost is instead minimized by driving the gripper toward the pose encoded in the goal latent, with the block left in place.

The collapse of test-time search is not an under-tuned-search artifact. The solvers run the LeWM's default search configuration, and each CEM step already evaluates 300 candidate action sequences over 30 iterations, driving the latent planning cost to near zero (Fig. 2) without moving the block. Scaling the search compute to 50,000 candidate rollouts per planning step, more than $5\times$ this default, leaves success at exactly 0% throughout (Appendix A). The limiter is the latent-distance objective and the learned predictor it is optimized against, not the search budget. GC-IDM is the best method on every task for both representations, and on Sub-JEPA latents it is dominant on the hardest one, with the smallest inflation gap on OGB-Cube ($+10.4$ vs. $+29.4$ for LeWM). What that advantage depends on is the subject of the next section. GC-IDM is handed the remaining horizon, the time-to-target, which a search planner must instead discover for itself.

5 The Critical Temporal Knowledge Assumption of Amortized Planning

We isolate the horizon input that the amortized policy's success depends on, the remaining horizon h, which encodes the distance to the goal. The GC-IDM controller exposes a single hyper-paramter T that serves as both the conditioning horizon, $h = \min(T - t, H_{\max})$, and the episode budget. T does not change $H_{\max}$, which stays fixed at the amortizer's training horizon of 50. What T decides is when the $\min$ binds. For $T \leq H_{\max}$ the horizon counts down from T to 1 from the first step. For $T > H_{\max}$ it is held at $H_{\max}$ for the first $T - H_{\max}$ steps and counts down only after that. We probe this two ways, with the true goal distance fixed at 50 on OGB-Cube.

Following GC-IDM's inference process, with T set to the episode budget, success peaks at T=50 (the true distance) and falls off either side (Fig. 3b). The clean, unconfounded signal comes from the over-stated side. At T=100 the agent has twice the budget it needs, yet Sub-JEPA falls from 84.8



(a) Normalized latent cost over each planning step and CEM iterations, cost reduced 99%.

(b) Normalized object distance to goal position, executing (a) plans.

(c) Normalized latent cost and object distance to goal position for GC-IDM executed trajectories.

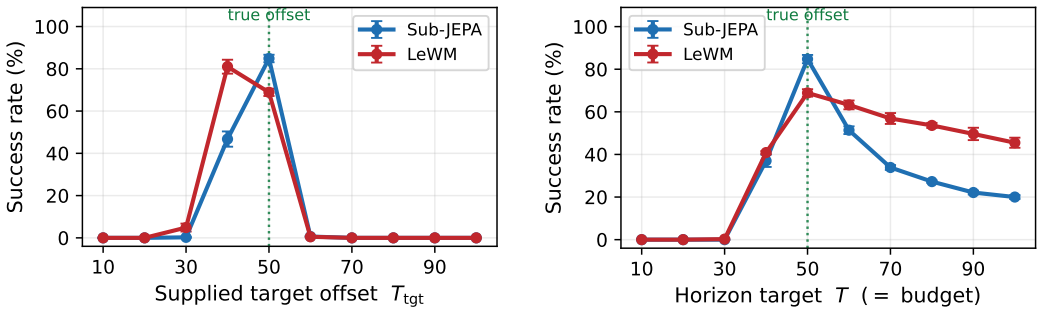
Figure 2: **A decreasing latent goal-cost does not by itself indicate task progress.** On OGB-Cube, with Sub-JEPA representations, evaluated from t=0, we show the mean over 12 episodes, shaded ± 1 standard deviation, with values normalized to their initial reading. CEM minimizes the latent objective it optimizes (a) without displacing the block (b), whereas under GC-IDM the same latent goal-cost $\|z_t - z_g\|$ falls only as the block physically approaches the goal (c).

Table 1: **Start-controlled evaluation across the benchmark suite.** Task success rate (%), 200 episodes/cell, mean ± std over 4 seeds, for both frozen representations. Here "rand." starts from a random mid-episode state and "infl." = rand. − t=0. From the controlled start (t=0), search collapses on the 3D manipulation task (every solver scores exactly 0% on OGB-Cube), while amortized GC-IDM (bold t=0) is the only method that solves it. On the 2D suites search retains partial success.

	Sub-JEPA			LeWM		
Method	t=0	rand.	infl.	t=0	rand.	infl.
OGB-Cube (3D manipulation)						
GC-IDM	**84.9** ± 1.8	95.2 ± 1.2	+10.4 ± 1.8	**68.9** ± 1.7	98.2 ± 0.6	+29.4 ± 1.2
CEM	0.0 ± 0.0	55.9 ± 4.0	+55.9 ± 4.0	0.0 ± 0.0	51.6 ± 5.3	+51.6 ± 5.3
iCEM	0.0 ± 0.0	59.4 ± 2.9	+59.4 ± 2.9	0.0 ± 0.0	52.0 ± 2.8	+52.0 ± 2.8
MPPI	0.0 ± 0.0	37.0 ± 1.5	+37.0 ± 1.5	0.0 ± 0.0	38.1 ± 1.3	+38.1 ± 1.3
SGD	0.0 ± 0.0	22.4 ± 2.4	+22.4 ± 2.4	0.0 ± 0.0	13.6 ± 1.2	+13.6 ± 1.2
Push-T (2D pushing)						
GC-IDM	**86.6** ± 1.8	83.6 ± 0.4	−3.0 ± 2.1	**85.2** ± 2.7	82.8 ± 2.5	−2.5 ± 3.9
CEM	19.1 ± 1.7	38.9 ± 2.9	+19.8 ± 3.7	17.5 ± 2.6	35.0 ± 2.3	+17.5 ± 4.3
iCEM	9.2 ± 0.6	32.8 ± 3.4	+23.5 ± 3.2	9.1 ± 1.5	27.4 ± 2.6	+18.2 ± 3.2
MPPI	14.2 ± 2.4	22.4 ± 2.9	+8.1 ± 4.5	10.8 ± 3.0	19.8 ± 3.8	+9.0 ± 3.8
SGD	0.2 ± 0.4	0.2 ± 0.2	+0.0 ± 0.6	0.0 ± 0.0	0.6 ± 0.5	+0.6 ± 0.5
Two-Room (2D navigation)						
GC-IDM	**100.0** ± 0.0	100.0 ± 0.0	+0.0 ± 0.0	**100.0** ± 0.0	100.0 ± 0.0	+0.0 ± 0.0
CEM	51.2 ± 3.3	53.9 ± 5.4	+2.6 ± 8.5	38.0 ± 2.7	47.8 ± 2.1	+9.8 ± 2.7
iCEM	56.4 ± 2.6	58.4 ± 5.2	+2.0 ± 7.1	43.6 ± 2.9	57.6 ± 3.5	+14.0 ± 2.6
MPPI	26.8 ± 4.4	29.5 ± 2.8	+2.8 ± 5.2	26.1 ± 1.3	29.0 ± 2.5	+2.9 ± 1.8
SGD	18.0 ± 2.6	20.1 ± 2.4	+2.1 ± 1.1	13.4 ± 2.0	16.2 ± 2.5	+2.9 ± 2.5
Reacher (2D reaching)						
GC-IDM	**99.6** ± 0.4	99.6 ± 0.4	+0.0 ± 0.6	**99.8** ± 0.2	99.5 ± 0.6	−0.2 ± 0.8
CEM	92.1 ± 0.5	75.8 ± 3.2	−16.4 ± 3.5	91.8 ± 2.1	76.9 ± 3.9	−14.9 ± 1.8
iCEM	93.4 ± 1.5	81.5 ± 2.5	−11.9 ± 3.8	93.8 ± 0.8	81.4 ± 1.4	−12.4 ± 0.7
MPPI	38.6 ± 2.3	38.2 ± 1.2	−0.4 ± 2.6	35.5 ± 1.8	34.8 ± 3.7	−0.8 ± 2.8
SGD	4.4 ± 0.4	4.6 ± 1.6	+0.2 ± 1.6	4.1 ± 0.6	4.0 ± 2.0	−0.1 ± 1.6

to 20.0 (LeWM 68.9→45.5). At that setting the policy is told the goal is $H_{\max}$ steps away for the first half of the episode, and only then does the countdown begin. Budget is not the limiter. The mis-set horizon is, which inverts the usual intuition that more budget only helps. The under-stated side collapses too, but that result is confounded, as T also sets the budget and the episode is then too short to reach a 50-step goal.

To isolate which of the horizon or budget does the damage, we hold the budget fixed at 50 steps, always adequate for a 50-step goal, and vary only the told horizon T_{tgt}, giving $h = \min(T_{\text{tgt}} - t, H_{\max})$ (Fig. 3a). Time is never the limiter here, and any failure is therefore attributable to the horizon input alone. The pinned budget also sharpens the effect, as an over-stated T_{tgt} leaves no episode after the clamp, and at T_{tgt}=100 the horizon never leaves $H_{\max}$. Mis-stating the distance in either direction



(a) *Decoupled diagnostic.* The budget is held fixed at an adequate 50 steps and only the told horizon T_{tgt} varies, and time is therefore never the limiter. Success peaks sharply at the true offset and collapses once the model is mis-told, isolating the horizon input as the cause.

(b) *GC-IDM inference protocol.* A single hyperparameter T sets both the target and the budget, making a wrong T simultaneously a wrong horizon and a wrong budget. Success peaks at the true offset and decays away from it, most steeply for Sub-JEPA.

Figure 3: **GC-IDM depends on the target distance** (OGB-Cube, start-controlled t=0, true offset 50, 4 seeds, error bars ± 1 standard deviation over seeds). Both panels vary the horizon the controller is told. In (b) the budget moves with it, while in (a) the budget is held fixed and adequate. Performance is maximized only at the true target distance and degrades once that value is mis-specified.

collapses GC-IDM, which peaks only at T_{tgt}=50 for Sub-JEPA, matching the true distance. The horizon input itself is what the method depends on. LeWM is more forgiving of a wrong horizon ($\sim$ 23 vs. Sub-JEPA's $\sim$ 65 points over T=50 $\rightarrow$ 100 in Fig. 3b) but mis-calibrated, peaking at T_{tgt}=40 rather than the true 50.

A sub-goal that is spatially plausible but time-miscalibrated still triggers the collapse. The horizon could in principle be supplied by a higher-order planner rather than an oracle. Given GC-IDM's sharp sensitivity to mis-specification, however, such a supplied estimate would have to be near-exact to help. Such a hierarchical planner would be usable by our proposed method VC-IDM (§6), which tolerates an imprecise horizon rather than requiring the near-exact value GC-IDM depends on.

6 Removing the Temporal Knowledge Dependence with VC-IDM

The horizon should be inferred, not supplied. We propose the Value-Conditioned Inverse Dynamics Model (VC-IDM), which learns a temporal-distance value $V(z_t, z_g)$ that regresses steps-to-go on the frozen latents, then conditions the inverse-dynamics model on the value's own estimate. At deployment the policy reads its horizon from V each step, with no privileged offset injected. It regresses steps-to-go and feeds the estimate as the horizon. Its gains are concentrated in the deployment-relevant regimes (unknown, variable, or out-of-range targets). With the horizon known and in-distribution it is competitive with GC-IDM.

Both networks operate on the frozen d=192 latents. The GC-IDM amortizer ($\sim$ 1.55M params) maps the concatenation $[z_t, z_g]$ through a 3-layer (512-wide, LayerNorm/GELU/Dropout) backbone with a linear action head. The normalized horizon is sinusoidally embedded and injected by zero-initialized AdaLN-Zero modulation (following the GC-IDM paper). The temporal-distance value V ($\sim$ 0.73M params) is a separate network with the same backbone architecture and its own weights, outputting a scalar (predicted steps-to-go).

We first pretrain V to regress steps-to-go on frozen-embedding triples (z_t, z_g, k) with z_g the latent k steps ahead, $k \sim \mathrm{U}[1, H_{\max}]$, then freeze it. We then train the amortizer on the frozen value's own estimate, $\hat{h} = \mathrm{round}(\mathrm{clip}(V(z_t, z_g), 1, H_{\max}))$, minimizing $\|\mathrm{IDM}(z_t, z_g, \hat{h}) - a\|^2$. Training on $\hat{h}$ rather than the exact k keeps training and deployment consistent. The amortizer acts under the same frozen, slightly-biased horizon signal it will receive at test time.

Evaluation regimes. We test three scenarios under the start-controlled protocol. The first is generalization beyond the training horizon (models trained at $H_{\max}$=50, evaluated at offset $H \in$

Table 2: **Horizon generalization and in-distribution behavior.** Success % (mean ± std over 4 seeds), best of the three methods per group in bold. *Generalization* uses models trained at $H_{\max}$=50, evaluated at offset H, and *In-distribution* uses models trained at $H_{\max}=H$. Beyond the training horizon GC-IDM collapses while VC-IDM holds, and with the horizon known and in-distribution the two are competitive. Columns are GC-IDM (GC), the horizon-free GC-IDM (hf), and VC-IDM (VC).

			Generalization ($H_{\max}$=50)			In-distribution ($H_{\max}=H$)		
Env	Repr	H	GC	hf	VC	GC	hf	VC
Cube	Sub-JEPA	50	**84.9** ± 1.8	42.0 ± 0.4	77.0 ± 2.4	**84.9** ± 1.8	42.0 ± 0.4	77.0 ± 2.4
		75	12.1 ± 1.9	18.5 ± 2.5	**29.4** ± 4.7	**73.9** ± 2.7	34.6 ± 1.6	67.9 ± 4.8
		100	4.9 ± 1.6	19.1 ± 2.5	**25.0** ± 2.4	**72.4** ± 2.5	42.6 ± 4.1	70.8 ± 2.6
	LeWM	50	68.9 ± 1.7	39.8 ± 3.4	**70.4** ± 3.0	68.9 ± 1.7	39.8 ± 3.4	**70.4** ± 3.0
		75	9.1 ± 0.4	30.8 ± 1.0	**60.5** ± 3.8	**95.1** ± 1.2	43.1 ± 1.1	78.5 ± 2.9
		100	3.5 ± 1.1	11.4 ± 1.6	**30.1** ± 1.0	**85.8** ± 1.8	58.2 ± 5.4	80.4 ± 1.0
Push-T	Sub-JEPA	50	86.0 ± 1.8	74.5 ± 3.4	**89.1** ± 1.5	86.0 ± 1.8	74.5 ± 3.4	**89.1** ± 1.5
		75	46.9 ± 2.2	30.5 ± 1.8	**67.8** ± 2.8	73.6 ± 2.6	55.6 ± 1.7	**78.5** ± 6.0
		100	25.6 ± 1.7	18.2 ± 1.9	**45.8** ± 1.8	65.2 ± 1.8	46.9 ± 1.9	**78.0** ± 1.7
	LeWM	50	85.9 ± 2.5	70.2 ± 2.5	**87.9** ± 2.2	85.9 ± 2.5	70.2 ± 2.5	**87.9** ± 2.2
		75	47.6 ± 3.2	32.2 ± 2.9	**63.0** ± 3.7	71.0 ± 2.2	50.5 ± 1.7	**75.9** ± 1.6
		100	22.9 ± 1.1	17.9 ± 2.8	**39.4** ± 0.7	59.5 ± 0.9	42.6 ± 2.6	**71.8** ± 0.9
Two-Room	Sub-JEPA	50	**100.0** ± 0.0	96.9 ± 1.1	**100.0** ± 0.0	**100.0** ± 0.0	96.9 ± 1.1	**100.0** ± 0.0
		75	**99.1** ± 0.6	97.9 ± 1.2	**99.1** ± 0.6	**100.0** ± 0.0	98.9 ± 0.5	**100.0** ± 0.0
		100	**95.2** ± 1.8	92.0 ± 1.7	94.9 ± 1.5	**99.8** ± 0.2	99.0 ± 0.0	**99.8** ± 0.2
	LeWM	50	**100.0** ± 0.0	96.4 ± 1.1	**100.0** ± 0.0	**100.0** ± 0.0	96.4 ± 1.1	**100.0** ± 0.0
		75	**100.0** ± 0.0	99.4 ± 0.5	**100.0** ± 0.0	**100.0** ± 0.0	99.0 ± 0.6	**100.0** ± 0.0
		100	**99.6** ± 0.2	97.9 ± 0.5	**99.6** ± 0.4	**100.0** ± 0.0	99.6 ± 0.2	**100.0** ± 0.0
Reacher	Sub-JEPA	50	99.6 ± 0.4	98.1 ± 0.4	**99.9** ± 0.2	99.6 ± 0.4	98.1 ± 0.4	**99.9** ± 0.2
		75	99.0 ± 0.4	99.2 ± 0.6	**99.5** ± 0.6	99.2 ± 0.2	97.8 ± 0.6	**99.6** ± 0.4
		100	**99.4** ± 0.2	99.0 ± 0.6	99.2 ± 0.6	**99.5** ± 0.4	98.2 ± 0.9	**99.5** ± 0.4
	LeWM	50	99.8 ± 0.2	98.1 ± 0.2	**99.9** ± 0.2	99.8 ± 0.2	98.1 ± 0.2	**99.9** ± 0.2
		75	**99.1** ± 0.2	99.0 ± 0.7	**99.1** ± 0.4	99.1 ± 0.2	97.4 ± 1.0	**99.4** ± 0.4
		100	99.2 ± 0.8	**99.4** ± 0.2	99.2 ± 0.8	**99.5** ± 0.4	98.4 ± 0.7	**99.5** ± 0.4

$\{50, 75, 100\}$). The second is an in-distribution long horizon ($H_{\max}$ matched to H). The third is randomized targets (per-episode offset drawn from $[0.5H, 1.5H]$). The full set of results appears in Tables 2 and 3.

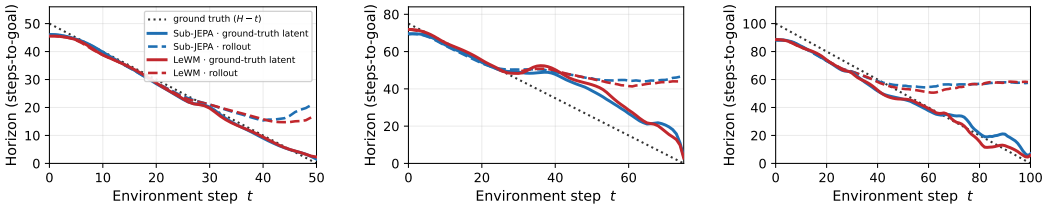
We report VC-IDM at the $4H$ budget and GC-IDM at H, which is each method at the budget that suits it. The two are not interchangeable. Extra budget helps VC-IDM, whose horizon is decoupled from it and read from V each step (Fig. 5), and hurts GC-IDM, whose budget is its horizon (Fig. 3b). When the target distance is randomized, the realistic deployment case (Table 3), VC-IDM at $4H$ beats GC-IDM at H in all twelve Cube and Push-T cells (e.g. Push-T $+27$ at H=50, Cube $+18$ at H=100). The gain is a budget effect the amortized baseline cannot access. Held to GC-IDM's own budget of H, VC-IDM does not lead, and the advantage appears once the budget is doubled. With goals at variable distances there is no single good budget for GC-IDM. Reaching the far goals requires a large budget, which pins its horizon at $H_{\max}$ and over-states the near ones, while a small budget cannot reach the far goals at all. VC-IDM sidesteps this by inferring each goal's horizon. The horizon-free variant (hf) is the weakest method in every randomized cell, and removing the horizon is worse than mis-setting it here. Beyond the training range the ordering reverses, and hf overtakes GC-IDM on Cube (Table 2).

Generalization and in-distribution behavior complete the picture (Table 2). With $H_{\max}$=50 models evaluated beyond their training range, GC-IDM collapses (Cube $12 \to 5\%$ at H=75, 100, Push-T $47 \to 26\%$) while VC-IDM degrades gracefully (Cube $29/25\%$, Push-T $68/46\%$), a 1.4 to $5\times$ gap. When the horizon is both known and in-distribution, our method is competitive with GC-IDM. It leads on Push-T in all six cells, by 2 to 13 points, and trails on Cube in five of six, by up to 17 points. The pattern is consistent across all three regimes. GC-IDM leads only when the distance is known and in-distribution, and the moment it is unknown, variable, or beyond the training range, VC-IDM wins by inferring it.

We investigate the value estimator's performance and how conditioning affects budget dependence, for both representations (Fig. 4 and 5). (i) Calibration (Fig. 4). On the ground-truth (expert) latent

Table 3: **Randomized target distance.** Per-episode offset $\in [0.5H, 1.5H]$, with columns as in Table 2. VC-IDM runs at budget $4H$ and GC-IDM at H, each at the budget that suits it (Fig. 3b and Fig. 5). VC-IDM wins in every Cube and Push-T cell. Two-Room appears only at H=50, as its episodes are too short for the band at higher H.

Env	Repr	H	GC	hf	VC
Cube	Sub-JEPA	50	48.4 ± 0.4	28.2 ± 2.3	$\mathbf{54.6} \pm 1.0$
		75	46.9 ± 2.9	35.1 ± 1.4	$\mathbf{62.9} \pm 3.7$
		100	52.9 ± 2.9	46.2 ± 3.4	$\mathbf{71.1} \pm 3.1$
	LeWM	50	55.8 ± 1.3	24.8 ± 2.0	$\mathbf{62.9} \pm 1.4$
		75	51.8 ± 3.5	41.5 ± 3.0	$\mathbf{68.2} \pm 4.9$
		100	62.4 ± 2.2	55.4 ± 4.8	$\mathbf{77.2} \pm 2.0$
Push-T	Sub-JEPA	50	60.6 ± 3.6	54.0 ± 1.7	$\mathbf{88.1} \pm 1.9$
		75	52.4 ± 3.3	44.4 ± 2.2	$\mathbf{74.0} \pm 5.0$
		100	45.4 ± 0.4	39.0 ± 2.9	$\mathbf{67.8} \pm 2.9$
	LeWM	50	59.8 ± 2.0	52.8 ± 1.1	$\mathbf{84.5} \pm 2.9$
		75	49.2 ± 3.9	40.6 ± 3.4	$\mathbf{72.6} \pm 7.4$
		100	40.6 ± 0.9	35.4 ± 2.2	$\mathbf{64.6} \pm 1.9$
Two-Room	Sub-JEPA	50	$\mathbf{100.0} \pm 0.0$	94.8 ± 1.6	$\mathbf{100.0} \pm 0.0$
	LeWM	50	$\mathbf{100.0} \pm 0.0$	93.5 ± 1.7	$\mathbf{100.0} \pm 0.0$
Reacher	Sub-JEPA	50	99.8 ± 0.2	97.2 ± 0.9	$\mathbf{99.9} \pm 0.2$
		75	$\mathbf{99.4} \pm 0.5$	96.6 ± 1.7	$\mathbf{99.4} \pm 0.5$
		100	99.0 ± 0.6	96.2 ± 1.3	$\mathbf{99.2} \pm 0.4$
	LeWM	50	99.8 ± 0.2	96.8 ± 1.3	$\mathbf{99.9} \pm 0.2$
		75	$\mathbf{99.4} \pm 0.4$	96.9 ± 1.8	99.2 ± 0.6
		100	99.1 ± 0.4	96.2 ± 1.2	$\mathbf{99.2} \pm 0.4$



(a) Offset H=50 ($H_{\max}$=H). (b) Offset H=75 ($H_{\max}$=H). (c) Offset H=100 ($H_{\max}$=H).

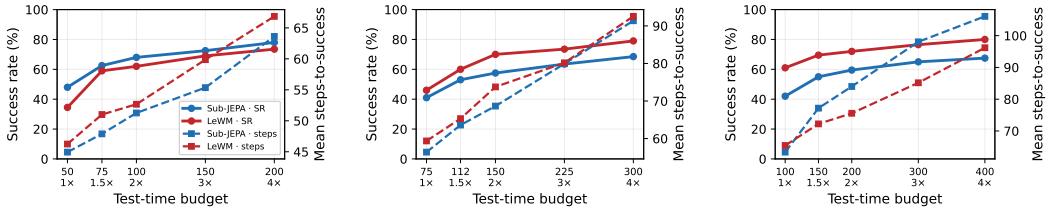
Figure 4: **Horizon-predictor calibration (OGB-Cube).** Per-step value estimate $\hat{h} = V(z_t, z_g)$ for Sub-JEPA and LeWM. Solid lines run along the ground-truth (expert) latent trajectory, dashed lines along the agent's VC-IDM rollout (mean over 200 episodes). On-manifold the value tracks the true remaining horizon $H-t$ (dotted). In the rollout it plateaus once the agent stalls off-manifold. The solid-dashed gap is off-manifold calibration loss, not regression error.

trajectory the learned value's estimate $\hat{h} = V(z_t, z_g)$ descends monotonically to zero at the goal for both representations, confirming it is a genuine distance estimate. Along the agent's own rollout, the estimate follows this until the agent leaves the data manifold, then plateaus. The plateau is an off-manifold artifact (the agent stalls at roughly constant latent distance from the goal), not a regression error, and is not closed by a better value loss, pointing to on-policy value data as the remaining lever. (ii) Budget (Fig. 5). Success rises monotonically with test-time budget at every offset for both representations, the mirror of GC-IDM (§5). The extra budget is spent reaching more, and farther, goals, and mean steps-to-success rises with it.

7 Discussion and Limitations

Two recommendations follow. Report from t=0, since a random start can hand a planner unearned success whose magnitude and sign depend on the task. And infer the horizon rather than supplying it, since coupling it to the budget both hides the dependence and makes the method fragile.

Finally, a note on why amortization works. We measure isotropy by the effective rank (participation ratio) of the latent covariance, $\mathrm{PR} = (\sum_i \lambda_i)^2 / \sum_i \lambda_i^2$ with λ_i the eigenvalues of that covariance,



(a) Offset H=50. (b) Offset H=75. (c) Offset H=100.

Figure 5: **Budget robustness (OGB-Cube).** Success rate (solid, left axis) and mean steps-to-success (dashed, right axis) vs. test-time budget ($1\times$ to $4\times$ the offset), for Sub-JEPA and LeWM. Success rises with budget at every offset. VC-IDM spends extra budget reaching more goals, the opposite of GC-IDM, whose success degrades with budget (§5).

which equals the full dimension (d=192) under perfect isotropy and collapses when variance concentrates. Sub-JEPA is strongly anisotropic ($\mathrm{PR} \approx 6/192$) yet gives the best amortized policy, beating the near-isotropic LeWM ($\mathrm{PR} \approx 100/192$). We report this as an observation, not a conclusion. Two representations cannot identify which geometric property makes a latent space amortizable, and a wider sweep would be needed to settle that.

Limitations. (i) Scoped external validity. We evaluate on four `stable-worldmodel` environments, the same suite the latent-world-model methods we build on are benchmarked on [7, 16, 8], which keeps the comparison faithful to how the field measures these controllers. Only one of the four is 3D, and navigation and reaching saturate, leaving the signal to two manipulation/pushing tasks. Using the standard suite isolates the mechanism cleanly. Transfer to a second latent-world-model ecosystem or to harder, multi-objective tasks remains untested. (ii) Test-time search failure. We establish that amortization beats search and pinpoint the horizon dependence it relies on, but a causal analysis of why search fails is out of scope. (iii) Dependency shift. VC-IDM trades a supplied horizon for a learned one, and success now rests on the value's accuracy. (iv) The IDM is behavior-cloned on offline expert data. Goal stitching and out-of-distribution goals, where a quasimetric value could help beyond cloning, are untested.

8 Conclusion

We revisited the claim that amortized inverse dynamics can replace test-time search for planning in latent world models. Under start-controlled evaluation, search under the standard latent-distance cost collapses on the manipulation task where planning should matter, and only the amortized policy solves it. That advantage, however, rests on being handed the target distance. Replacing the supplied horizon with a learned temporal-distance value (VC-IDM) removes the dependence and wins where a deployed system operates (unknown, variable, or out-of-range targets), improving with budget rather than degrading. Next steps are external validity on a second ecosystem, and on-policy training of the value on the agent's own rollout states, which the diagnostics suggest would curb the off-manifold drift.

References

[1] Pulkit Agrawal, Ashvin Nair, Pieter Abbeel, Jitendra Malik, and Sergey Levine. Learning to poke by poking: Experiential learning of intuitive physics. *arXiv preprint arXiv:1606.07419*, 2016.

[2] Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv:2206.11795*, 2022.

[3] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. *arXiv preprint arXiv:1809.01999*, 2018.

[4] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[5] Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022.

[6] Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation. *arXiv preprint arXiv:2605.21800*, 2026.

[7] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[8] Hoang Nguyen, Xiaohao Xu, and Xiaonan Huang. Latent geometry beyond search: Amortizing planning in world models. *arXiv preprint arXiv:2605.08732*, 2026.

[9] Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. Ogbench: Benchmarking offline goal-conditioned rl. *arXiv preprint arXiv:2410.20092*, 2024.

[10] Keiran Paster, Sheila A. McIlraith, and Jimmy Ba. Planning from pixels using inverse dynamics models. *arXiv preprint arXiv:2012.02419*, 2020.

[11] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. *arXiv preprint arXiv:1705.05363*, 2017.

[12] Cristina Pinneri, Shambhuraj Sawant, Sebastian Blaes, Jan Achterhold, Joerg Stueckler, Michal Rolinek, and Georg Martius. Sample-efficient cross-entropy method for real-time planning. *arXiv preprint arXiv:2008.06389*, 2020.

[13] Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology And Computing In Applied Probability*, 1(2):127–190, 1999. ISSN 1573-7713. doi: 10.1023/a:1010091220143. URL `http://dx.doi.org/10.1023/A:1010091220143`.

[14] Tongzhou Wang, Antonio Torralba, Phillip Isola, and Amy Zhang. Optimal goal-reaching reinforcement learning via quasimetric learning. *arXiv preprint arXiv:2304.01203*, 2023.

[15] Grady Williams, Andrew Aldrich, and Evangelos Theodorou. Model predictive path integral control using covariance variable importance sampling. *arXiv preprint arXiv:1509.01149*, 2015.

[16] Kai Zhao, Dongliang Nie, Yuchen Lin, Zhehan Luo, Yixiao Gu, Deng-Ping Fan, and Dan Zeng. Sub-jepa: Subspace gaussian regularization for stable end-to-end world models. *arXiv preprint arXiv:2605.09241*, 2026.

Appendix A CEM Hyperparameter Sweep (OGB-Cube)

To confirm that the start-controlled search collapse (§4) is not an artifact of an under-provisioned planner, we sweep the CEM search budget over a grid. We vary the candidates per iteration (`num_samples` $\in \{300, 500, 1000\}$), the number of refinement iterations (`n_steps` $\in \{10, 30, 50\}$), and the planning/receding horizon ($\in \{5, 10\}$), giving 18 configurations. All other settings match the latent-cost diagnostic (OGB-Cube, Sub-JEPA, start-controlled t=0, offset 50, budget 100, over 12 episodes). This spans the search compute from $300 \times 10 = 3{,}000$ candidate rollouts per planning step to $1000 \times 50 = 50{,}000$, a $16\times$ range that contains the 9,000-rollout default used throughout (§1).

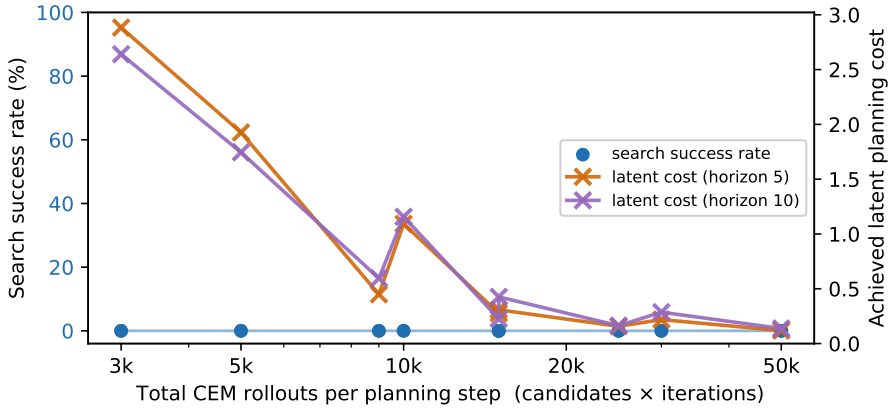


Figure 6: **Search stays collapsed as compute scales** (OGB-Cube, Sub-JEPA, start-controlled t=0). Across all 18 configurations, search success is exactly 0% and net block displacement is 0.0 cm (max 0.001 cm), while the achieved latent planning cost the optimizer minimizes falls from 2.9 to 0.12 as compute grows. More search buys a better-optimized latent objective, not task progress.

Across every configuration, search success remains exactly 0% and the cube's net displacement stays at 0.0 cm (Fig. 6). Additional compute only lets the optimizer drive the latent goal-cost it minimizes lower (from 2.9 down to 0.12), reinforcing the reading of §4. On this task the latent-distance objective and the frozen predictor are what cap performance.